MoA Prediction — Inference Notebook

Kaggle competition: Mechanisms of Action (MoA) Prediction

This notebook produces `submission.csv` **without any training**. It loads the models and preprocessing pipelines trained by the training notebook (`moa_prediction.ipynb`) from a Kaggle Dataset of artifacts and applies them to the test set. Training is not repeated here (about 130 minutes on 2x NVIDIA T4), and this notebook is meant to run offline, as required by a Kaggle code competition.

Inputs

Add both as notebook inputs (*Add Input*):

Input	Content
Competition data `lish-moa`	`test_features.csv`, `sample_submission.csv`
Dataset of artifacts (`moa-artifacts`)	30 network checkpoints `best_*_fold_*_seed*.pth`, 5 `fold_pipeline_*.joblib`, 5 `fold_pipeline_raw_*.joblib`, 5 `catboost_fold_*.cbm`, 5 `catboost_fold_*_meta.joblib`, and `metadata.json` (51 files)

Both folders are **located automatically by file name** (Step 1.2), so the notebook does not depend on the mount path or on the dataset owner's user name. The artifacts folder is read-only: nothing is written there. The only output is `/kaggle/working/submission.csv`.

Prerequisites

- Internet **off** (no `pip install` is needed).
- A GPU is recommended: the FT-Transformer is slow on CPU. A CPU-only run still works.
- Library versions should match the training environment: the fold pipelines are scikit-learn pickles. Step 9.1 prints the installed versions.

Workflow

Section headers and in-code comments carry the matching step number (`# Step N.k`). Steps 3 to 8 reuse the training notebook code with three differences:

- the pipeline and CatBoost functions read from ARTIFACT_DIR (argument artifact_dir) instead of WORK_DIR;
- sanity_check_submission also accepts a test set without control rows;
- all training-only code is removed. The numerical path is the same, as confirmed by the 1.1e-16 consistency check.

1. **Setup** — imports, input-folder discovery, constants
   - 1.1 imports
   - 1.2 locate the artifacts and competition-data folders
   - 1.3 constants and hardware configuration
2. **Metadata, artifact and version checks** — load `metadata.json` , define the integrity check
3. **Feature engineering** — row statistics, one-hot encoding (same as training)
4. **Model architectures** — TabularResNet, FT-Transformer (needed to rebuild the networks before loading the weights)
5. **Preprocessing** — apply the already-fitted fold pipelines (RankGauss + PCA, RankGauss only)
6. **Neural-network inference** — 2 models x 3 seeds x 5 folds = 30 checkpoints, averaged
7. **CatBoost inference** — 5 fold models, averaged
8. **Submission building and sanity checks** — control rows set to 0.0, shape/NaN/inf checks
9. **Main pipeline** — runs steps 2 to 8 end to end and writes `submission.csv`
   - 9.1 check the artifacts, load the test data
   - 9.2 feature engineering on treated rows
   - 9.3 apply the fold pipelines
   - 9.4 neural-network and CatBoost inference, weighted blend
   - 9.5 build, check and export the submission

Notes

- **No refitting.** Every transformation is applied with the pipeline fitted on the training folds. Test rows never influence the preprocessing.
- **Final prediction.** Weighted blend of ResNet, FT-Transformer and CatBoost, with the weights found by the nested cross-validation in the training notebook (stored in `metadata.json`).
- **Consistency check.** On the public test set, the predictions of this notebook were compared with those of the training notebook: maximum absolute difference 1.1e-16.

1. Setup: imports, input folders and constants

Imports, automatic discovery of the input folders, and the constants that must match the training notebook.

In [1]:
```python
# Step 1.1 - imports
import os
import glob
import json
import math
import time
import warnings
import sys

# Set before importing PyTorch to reduce GPU memory fragmentation
# when multiple parallel processes allocate tensors.
os.environ.setdefault("PYTORCH_CUDA_ALLOC_CONF", "expandable_segments:True")

import numpy as np
import pandas as pd
import joblib
import sklearn
import catboost
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from joblib import Parallel, delayed
from joblib.externals.loky import get_reusable_executor
```

In [2]:
```python
# Step 1.2 - locate the input folders
def find_input_directory(file_name):
    """Returns the single folder under /kaggle/input that contains `file_name`.
    Searching by file name avoids hard-coded mount paths, which change between
    Kaggle platform versions and contain the dataset owner's user name."""
    matches = glob.glob(f"/kaggle/input/**/{file_name}", recursive=True)
    directories = sorted({os.path.dirname(path) for path in matches})
    if len(directories) != 1:
        raise FileNotFoundError(
            f"Expected exactly one folder containing '{file_name}' under /kaggle/input, "
            f"found {len(directories)}: {directories}. Check the notebook inputs "
            "(competition data + artifacts dataset).")
    return directories[0]
```

```
# For a run outside Kaggle, replace these three lines with explicit local paths.
ARTIFACT_DIR = find_input_directory("metadata.json")    # models, fold pipelines, metadata (read-only)
DATA_DIR = find_input_directory("test_features.csv")    # competition data (test set, sample submission)
OUTPUT_DIR = "/kaggle/working"                          # submission.csv is written here

print(f"Artifacts: {ARTIFACT_DIR}")
print(f"Data:      {DATA_DIR}")
```

```
Artifacts: /kaggle/input/datasets/marcomerlo37/moa-artifacts
Data:      /kaggle/input/competitions/lish-moa
```

In [3]:
```
# Step 1.3 - constants and hardware configuration
# Constants that must match the training notebook
CLIP_EPS = 1e-15               # probability clipping, avoids log(0) in the metric
CTL_LABEL = "ctl_vehicle"      # cp_type value of control rows (no mechanism of action)

# Inference hardware. Jobs are spread round-robin over the GPUs, 2 per GPU
# (overlaps compute and launch overhead). Without a GPU a single CPU worker is
# used: slower, but the result is the same.
N_GPUS = torch.cuda.device_count()
NN_JOBS_PER_GPU = 2
N_JOBS_NN = max(1, N_GPUS * NN_JOBS_PER_GPU)

# Inference batch sizes. The FT-Transformer is much smaller than the ResNet one:
# self-attention costs O(L^2) in the number of tokens (~890 per row), so a large
# batch would run out of GPU memory.
INFERENCE_BATCH_SIZE_BY_MODEL = {"resnet": 1024, "ft_transformer": 128}
DEFAULT_INFERENCE_BATCH_SIZE = 256

print(f"GPUs detected: {N_GPUS} | parallel NN jobs: {N_JOBS_NN}")
```

```
GPUs detected: 2 | parallel NN jobs: 4
```

2. Metadata and artifact check

Loads `metadata.json`, written by the training notebook (Step 14.9). It stores the exact feature order, the column groups, the target names and the blend weights. The feature order matters: the fold pipelines select columns by position, so a different order would silently feed the wrong columns to the models. The integrity check is defined here and run in Step 9.1.

In [4]:
```
# Step 2.1 - load the metadata exported by the training notebook
with open(f"{ARTIFACT_DIR}/metadata.json") as metadata_file:
    metadata = json.load(metadata_file)

feature_cols = metadata["feature_cols"]         # model input columns, in training order
g_cols = metadata["g_cols"]                     # gene-expression columns ('g-...')
```

```
c_cols = metadata["c_cols"]                    # cell-viability columns ('c-...')
target_cols = metadata["target_cols"]          # the 206 MoA targets
weights = metadata["weights"]                  # blend weights from the nested CV
# Fallbacks are the values used in the training run
N_FOLDS = metadata.get("n_folds", 5)
SEEDS = metadata.get("seeds", [42, 202, 2024])

ALL_MODEL_NAMES = ("resnet", "ft_transformer", "catboost")
assert set(weights) == set(ALL_MODEL_NAMES), f"Unexpected weight keys: {sorted(weights)}"
assert abs(sum(weights.values()) - 1.0) < 1e-6, "Blend weights must sum to 1"

print(f"{len(feature_cols)} features | {len(target_cols)} targets | "
      f"{N_FOLDS} folds | seeds {SEEDS}")
print("Blend weights: " + " | ".join(f"{name} {weights[name]:.4f}" for name in ALL_MODEL_NAMES))
```

```
887 features | 206 targets | 5 folds | seeds [42, 202, 2024]
Blend weights: resnet 0.4529 | ft_transformer 0.3746 | catboost 0.1725
```

In [5]:

```
# Step 2.2 - artifact integrity check
def list_expected_artifacts(n_folds, seeds, nn_model_names):
    """File names the inference needs, following the training notebook's naming."""
    names = []
    for fold in range(n_folds):
        names += [f"fold_pipeline_{fold}.joblib",
                  f"fold_pipeline_raw_{fold}.joblib",
                  f"catboost_fold_{fold}.cbm",
                  f"catboost_fold_{fold}_meta.joblib"]
        names += [f"best_{model_name}_fold_{fold}_seed{seed}.pth"
                  for model_name in nn_model_names for seed in seeds]
    return names


def check_artifacts(artifact_dir, n_folds, seeds, nn_model_names):
    """Fails early, with a clear message, if any artifact is missing (for example
    a file left out when the Dataset was created)."""
    expected = list_expected_artifacts(n_folds, seeds, nn_model_names)
    missing = [name for name in expected
               if not os.path.exists(os.path.join(artifact_dir, name))]
    if missing:
        raise FileNotFoundError(
            f"{len(missing)} of {len(expected)} artifact files missing in "
            f"{artifact_dir}. First missing: {missing[:5]}")
    print(f"Artifacts OK: {len(expected)} files found.")
```

In [6]:

```
# Step 2.3 - compare installed versions with those recorded by the training run
installed_versions = {"python": sys.version.split()[0], "scikit-learn": sklearn.__version__,
                      "torch": torch.__version__.split("+")[0], "catboost": catboost.__version__,
                      "numpy": np.__version__}
```

```python
for name, installed in installed_versions.items():
    trained = metadata.get("versions", {}).get(name)
    if trained is not None and trained != installed:
        print(f"WARNING {name}: training {trained} vs installed {installed}")
```

```
WARNING python: training 3.12.13 vs installed 3.13.15
WARNING torch: training 2.10.0+cu128 vs installed 2.11.0
WARNING numpy: training 2.0.2 vs installed 2.1.3
```

3. Feature engineering

Same transformations as in training: per-row summary statistics of the `g-` and `c-` blocks and one-hot encoding of `cp_time` / `cp_dose`. The test columns are re-aligned to the training columns.

In [7]:
```python
# Step 3.1 - row-wise summary statistics
def add_row_statistics(df, g_cols, c_cols):
    """
    Sum, mean, standard deviation, kurtosis and skewness are extracted separately
    for the g- and c- feature groups to provide the model with summary information on
    general activation.
    """
    df = df.copy()
    df["g_sum"] = df[g_cols].sum(axis=1)
    df["g_mean"] = df[g_cols].mean(axis=1)
    df["g_std"] = df[g_cols].std(axis=1)
    df["g_kurtosis"] = df[g_cols].kurtosis(axis=1)
    df["g_skew"] = df[g_cols].skew(axis=1)

    df["c_sum"] = df[c_cols].sum(axis=1)
    df["c_mean"] = df[c_cols].mean(axis=1)
    df["c_std"] = df[c_cols].std(axis=1)
    df["c_kurtosis"] = df[c_cols].kurtosis(axis=1)
    df["c_skew"] = df[c_cols].skew(axis=1)
    return df


# Step 3.2 - one-hot encoding of cp_time / cp_dose
def encode_categoricals(df, dummy_cols, reference_columns=None):
    """One-hot encodes cp_time/cp_dose, aligned to the training columns: a category
    missing from the test set becomes an all-zero column, and the column order is
    forced to match the training order."""
    df = pd.get_dummies(df, columns=dummy_cols, dtype=np.float64)
    if reference_columns is not None:
        df = df.reindex(columns=reference_columns, fill_value=0)
    return df
```

4. Model architectures

The network classes are needed to rebuild the models before loading the trained weights. They must be identical to the training notebook, otherwise `load_state_dict` fails or loads the weights into the wrong layers.

- `TabularResNet`
- `FTTransformer` (with its `FeatureTokenizer`)

In [8]:
```python
# Step 4.1 - Tabular ResNet
class TabularResNet(nn.Module):
    name = "resnet"
```

```
    def __init__(self, input_dim, output_dim=206, hidden_dim=512, dropout_rate=0.2):
        super().__init__()
        self.in_proj = nn.Sequential(
            nn.BatchNorm1d(input_dim),
            nn.Dropout(dropout_rate),
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
        )
        self.block1 = self._make_block(hidden_dim, dropout_rate)
        self.block2 = self._make_block(hidden_dim, dropout_rate)
        self.out_proj = nn.Sequential(
            nn.SiLU(), nn.Dropout(dropout_rate), nn.Linear(hidden_dim, output_dim)
        )

    @staticmethod
    def _make_block(hidden_dim, dropout_rate):
        return nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
            nn.Dropout(dropout_rate),
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
        )

    def forward(self, x):
        x = self.in_proj(x)
        x = F.silu(x + self.block1(x))
        x = F.silu(x + self.block2(x))
        return self.out_proj(x)


# Step 4.2 - FT-Transformer: feature tokenizer
class FeatureTokenizer(nn.Module):
    """
    Projects each scalar numeric feature into a token vector of dimension
    d_token via an independent linear transform.
    """

    def __init__(self, num_features, d_token):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(num_features, d_token))
        self.bias = nn.Parameter(torch.empty(num_features, d_token))
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        nn.init.zeros_(self.bias)
```

```
    def forward(self, x):
        # x shape: (Batch, Features)
        # broadcast: (Batch, Features, 1) * (Features, d_token) -> (Batch, Features, d_token)
        return x.unsqueeze(-1) * self.weight + self.bias


# Step 4.3 - FT-Transformer: encoder and head
class FTTransformer(nn.Module):
    """
    Simplified FT-Transformer variant (inspired by Gorishniy et al., 2021).
    Turns the tabular input into a sequence of tokens (one per feature),
    prepends a global [CLS] token, and applies multi-head self-attention.
    """

    name = "ft_transformer"

    def __init__(self, input_dim, output_dim=206, d_token=64, n_blocks=2,
                 attention_heads=4, dropout=0.2):
        super().__init__()
        self.tokenizer = FeatureTokenizer(input_dim, d_token)

        # Global [CLS] token
        self.cls_token = nn.Parameter(torch.zeros(1, 1, d_token))
        nn.init.normal_(self.cls_token, std=0.02)

        # Transformer blocks (GELU is natively supported and is the standard choice)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token,
            nhead=attention_heads,
            dim_feedforward=d_token * 4,
            dropout=dropout,
            activation="gelu",
            batch_first=True,
        )
        # enable_nested_tensor=False: no padding mask is used, and this avoids
        # a PyTorch UserWarning at construction time.
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_blocks,
                                                 enable_nested_tensor=False)

        self.layer_norm = nn.LayerNorm(d_token)
        self.head = nn.Sequential(
            nn.Dropout(dropout),
            nn.Linear(d_token, output_dim)
        )

    def forward(self, x):
        # Tokenization: (B, input_dim) -> (B, input_dim, d_token)
```

```
        tokens = self.tokenizer(x)

        # Prepend [CLS] token: (B, 1 + input_dim, d_token)
        cls_tokens = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat((cls_tokens, tokens), dim=1)

        # Transform via self-attention
        x = self.transformer(x)

        # Extract the output at the [CLS] token position (index 0)
        cls_out = self.layer_norm(x[:, 0, :])

        # Multi-label classification head
        return self.head(cls_out)
```

5. Preprocessing: fold pipelines (transform only)

Two preprocessing variants were fitted per fold during training and saved as `.joblib` files. Here they are only **applied**:

- with PCA (`fold_pipeline_*.joblib`), used by the ResNet;
- without PCA (`fold_pipeline_raw_*.joblib`), used by the FT-Transformer and CatBoost.

Each fold's models must receive the features produced by that same fold's pipeline.

In [9]:
```
# Step 5.1 - PCA pipeline (ResNet): VarianceThreshold -> RankGauss -> PCA
def apply_fold_pipeline(X, fold, artifact_dir=ARTIFACT_DIR):
    """Applies the pipeline already fitted for that fold (pure transform, no fitting).
    Output: RankGauss features + 50 PCA components of the g- block + 15 PCA
    components of the c- block + raw one-hot columns."""
    artifacts = joblib.load(f"{artifact_dir}/fold_pipeline_{fold}.joblib")
    X_vt = artifacts["selector"].transform(X)
    X_cont = X_vt[:, artifacts["cont_idx"]]
    X_dummy = X_vt[:, artifacts["dummy_idx"]]
    X_tf = artifacts["quantile_transformer"].transform(X_cont)
    pca_g_out = artifacts["pca_g"].transform(X_tf[:, artifacts["g_idx_in_cont"]])
    pca_c_out = artifacts["pca_c"].transform(X_tf[:, artifacts["c_idx_in_cont"]])
    return np.hstack([X_tf, pca_g_out, pca_c_out, X_dummy])


# Step 5.2 - raw pipeline, no PCA (FT-Transformer, CatBoost): VarianceThreshold -> RankGauss
def apply_fold_pipeline_raw(X, fold, artifact_dir=ARTIFACT_DIR):
    """Pure transform with the fold's raw pipeline. Keeps one column per original
    feature, which the FT-Transformer (one token per feature) and the CatBoost
    splits rely on. The one-hot columns are left untransformed."""
```

```
    artifacts = joblib.load(f"{artifact_dir}/fold_pipeline_raw_{fold}.joblib")
    X_vt = artifacts["selector"].transform(X)
    X_cont = X_vt[:, artifacts["cont_idx"]]
    X_dummy = X_vt[:, artifacts["dummy_idx"]]
    X_tf = artifacts["quantile_transformer"].transform(X_cont)
    return np.hstack([X_tf, X_dummy])
```

6. Neural-network inference

For each of the 30 checkpoints (2 models x 3 seeds x 5 folds): rebuild the network, load the weights, predict the test set in mini-batches. Jobs run in parallel with `joblib` (`loky` backend), distributed round-robin over the GPUs. Predictions are averaged over folds and seeds, separately for each model.

In [10]:
```
# Step 6.1 - model registry and helpers
MODEL_CLASSES = {"resnet": TabularResNet, "ft_transformer": FTTransformer}
MODEL_NAMES = tuple(MODEL_CLASSES.keys())


def shutdown_worker_pool():
    """Shuts down the loky worker pool so that idle workers release their CUDA
    memory. The informational loky warning "A worker stopped while some jobs
    were given to the executor" is raised by this shutdown itself, when all jobs
    are already done, so only that specific message is silenced here."""
    with warnings.catch_warnings():
        warnings.filterwarnings(
            "ignore", message="A worker stopped while some jobs were given")
        get_reusable_executor().shutdown(wait=True)


def _build_model(model_name, input_dim, output_dim):
    return MODEL_CLASSES[model_name](input_dim=input_dim, output_dim=output_dim)


def _resolve_device(gpu_id):
    """Single place for the GPU-assignment logic: jobs are distributed
    round-robin across all available GPUs (rather than converging on a
    single one), so the GPUs are used evenly. Falls back to the CPU."""
    if gpu_id is not None and N_GPUS > 0:
        torch.cuda.set_device(gpu_id)
        return torch.device(f"cuda:{gpu_id}")
    return torch.device("cpu")


# Step 6.2 - batched probability prediction
```

```
def predict_probabilities(model, X, device, batch_size, use_amp=False):
    """Returns sigmoid probabilities for X, computed in mini-batches."""
    model.eval()
    loader = DataLoader(TensorDataset(torch.tensor(X, dtype=torch.float32)),
                        batch_size=batch_size, shuffle=False)
    prob_chunks = []
    with torch.no_grad():
        for (x_batch,) in loader:
            x_batch = x_batch.to(device, non_blocking=True)
            with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
                logits = model(x_batch)
            prob_chunks.append(torch.sigmoid(logits.float()).cpu().numpy())
    # float64: keeps the later clipping to [1e-15, 1 - 1e-15] effective
    return np.vstack(prob_chunks).astype(np.float64)
```

In [11]:

```
# Step 6.3 - single test-set inference job (runs in a worker process)
def _predict_nn_job(fold, model_name, seed, X_test_final, num_targets, gpu_id,
                    artifact_dir=ARTIFACT_DIR, batch_size=None):
    device = _resolve_device(gpu_id)

    model = _build_model(model_name, X_test_final.shape[1], num_targets)
    checkpoint_path = f"{artifact_dir}/best_{model.name}_fold_{fold}_seed{seed}.pth"
    model.load_state_dict(torch.load(checkpoint_path, map_location=device, weights_only=True))
    model = model.to(device).eval()

    # Inference is run in mini-batches rather than a single monolithic forward
    # pass: for the FT-Transformer, one attention matrix of shape
    # (n_test_rows x heads x seq_len x seq_len) with seq_len ~890 would need
    # tens of GB in a single pass, far beyond the VRAM of a T4.
    # Full fp32 precision (no mixed precision), as in the training notebook's
    # test-set inference.
    inference_batch_size = batch_size or INFERENCE_BATCH_SIZE_BY_MODEL.get(
        model_name, DEFAULT_INFERENCE_BATCH_SIZE)
    test_preds = predict_probabilities(model, X_test_final, device, inference_batch_size,
                                       use_amp=False)

    del model
    if device.type == "cuda":
        torch.cuda.empty_cache()
    return model_name, test_preds


# Step 6.4 - test-set inference, averaged over folds and seeds
def predict_test_set_nn(X_test_final_by_fold_pca, X_test_final_by_fold_raw, num_targets,
                        n_folds=N_FOLDS, seeds=SEEDS, n_jobs=N_JOBS_NN, model_names=MODEL_NAMES):
    """
    Averages test-set predictions across all folds and seeds for each
    neural-network model. Each model receives the test features from the
```

```
    preprocessing variant it was trained on: PCA features for resnet,
    raw RankGauss features (no PCA) for the FT-Transformer.
    """
    n_samples = next(iter(X_test_final_by_fold_pca.values())).shape[0]
    preds = {name: np.zeros((n_samples, num_targets)) for name in model_names}
    n_checkpoints = n_folds * len(seeds)    # checkpoints per model

    jobs = [(fold, model_name, seed)
            for fold in range(n_folds)
            for model_name in model_names
            for seed in seeds]

    def _test_features(fold, model_name):
        return (X_test_final_by_fold_raw[fold] if model_name == "ft_transformer"
                else X_test_final_by_fold_pca[fold])

    results = Parallel(n_jobs=n_jobs, backend="loky")(
        delayed(_predict_nn_job)(
            fold, model_name, seed, _test_features(fold, model_name), num_targets,
            gpu_id=(job_number % N_GPUS) if N_GPUS > 0 else None,
        )
        for job_number, (fold, model_name, seed) in enumerate(jobs)
    )
    shutdown_worker_pool()

    for model_name, job_preds in results:
        preds[model_name] += job_preds / n_checkpoints

    return preds
```

7. CatBoost inference

One CatBoost model per fold, trained on the raw (non-PCA) features. Targets that were constant in a training fold were excluded from the fit; for those, the stored smoothed prior is used instead of a model output (see `catboost_fold_*_meta.joblib`). Predictions are averaged over the 5 folds.

In [12]:
```
# Step 7.1 - CatBoost test-set inference
def predict_test_set_catboost(X_test_raw, num_targets, n_folds=N_FOLDS, artifact_dir=ARTIFACT_DIR):
    from catboost import CatBoostClassifier

    n_samples = X_test_raw.shape[0]
    preds = np.zeros((n_samples, num_targets))
    for fold in range(n_folds):
        X_test_final = apply_fold_pipeline_raw(X_test_raw, fold, artifact_dir=artifact_dir)
        meta = joblib.load(f"{artifact_dir}/catboost_fold_{fold}_meta.joblib")
```

```
        active_cols, constant_cols, constant_vals = (
            meta["active_cols"], meta["constant_cols"], meta["constant_vals"]
        )

        model = CatBoostClassifier()
        model.load_model(f"{artifact_dir}/catboost_fold_{fold}.cbm")

        fold_preds = np.zeros((n_samples, num_targets))
        if len(active_cols) > 0:
            fold_preds[:, active_cols] = model.predict_proba(X_test_final)
        if len(constant_cols) > 0:
            fold_preds[:, constant_cols] = constant_vals[constant_cols]

        preds += np.clip(fold_preds, CLIP_EPS, 1 - CLIP_EPS) / n_folds
    return preds
```

8. Submission building and sanity checks

Definition of the functions that build and validate `submission.csv`. Control rows (`cp_type == "ctl_vehicle"`) are set to 0.0 as per the competition rules, because by definition they have no mechanism of action.

In [13]:
```
# Step 8.1 - build the submission DataFrame
def build_submission(raw_test, sample_submission, target_cols, trt_preds, trt_mask):
    """
    Places the model predictions on the treated test rows and explicitly sets
    the control-vehicle rows to 0.0 (they never have a mechanism of action),
    which lowers the log loss.
    """
    # Row order and columns of the test set and of the sample submission must match
    assert (sample_submission["sig_id"].values == raw_test["sig_id"].values).all(), \
        "sample_submission and test_features are not in the same row order"
    assert list(sample_submission.columns[1:]) == list(target_cols), \
        "Target columns differ from the sample submission"

    n_samples = len(raw_test)
    num_targets = len(target_cols)

    full_preds = np.zeros((n_samples, num_targets), dtype=np.float64)
    trt_indices = raw_test[trt_mask].index.values
    full_preds[trt_indices] = trt_preds
    full_preds = np.clip(full_preds, CLIP_EPS, 1.0 - CLIP_EPS)

    submission = sample_submission.copy()
    submission[target_cols] = full_preds
```

```
    submission.loc[raw_test["cp_type"] == CTL_LABEL, target_cols] = 0.0
    return submission


# Step 8.2 - sanity checks before export
def sanity_check_submission(submission, sample_submission, raw_test, target_cols):
    """
    Validates shape and columns, checks for NaN and infinite values, and
    ensures the control rows are exactly 0.0 before export, preventing
    submission failures on Kaggle.
    """
    assert submission.shape == sample_submission.shape, f"Wrong shape: {submission.shape}"
    assert list(submission.columns) == list(sample_submission.columns), "Column mismatch"
    assert submission.isna().sum().sum() == 0, "NaNs present"
    assert np.isinf(submission[target_cols].values).sum() == 0, "Infinite values present"

    ctl_ids = raw_test.loc[raw_test["cp_type"] == CTL_LABEL, "sig_id"]
    ctl_preds = submission[submission["sig_id"].isin(ctl_ids)][target_cols].values
    assert ctl_preds.size == 0 or ctl_preds.max() == 0.0, \
        f"ctl_vehicle rows not zeroed: max={ctl_preds.max()}"

    trt_preds = submission[~submission["sig_id"].isin(ctl_ids)][target_cols].values
    print(f"Min trt_cp probability: {trt_preds.min():.2e} | Max: {trt_preds.max():.2e} | Mean: {trt_preds.mean():.5f}")
    print("Sanity check passed.")
```

9. Main pipeline

Execution of steps 2 to 8 in order, to generate `submission.csv` in `OUTPUT_DIR`.

In [14]:
```
# Step 9.1 - check the artifacts, print library versions, load the test data
check_artifacts(ARTIFACT_DIR, N_FOLDS, SEEDS, MODEL_NAMES)
print(f"torch {torch.__version__} | scikit-learn {sklearn.__version__} | "
      f"catboost {catboost.__version__} | numpy {np.__version__}")

raw_test = pd.read_csv(f"{DATA_DIR}/test_features.csv")
sample_submission = pd.read_csv(f"{DATA_DIR}/sample_submission.csv")
print(f"Test rows: {len(raw_test)}")
```

```
Artifacts OK: 50 files found.
torch 2.11.0+cu128 | scikit-learn 1.6.1 | catboost 1.2.10 | numpy 2.1.3
Test rows: 3982
```

In [15]:
```
# Step 9.2 - feature engineering on treated rows only (control rows are set to 0.0 in Step 9.5)
trt_mask_test = raw_test["cp_type"] != CTL_LABEL
test_trt = raw_test[trt_mask_test].reset_index(drop=True)
```

```
test_trt = add_row_statistics(test_trt, g_cols, c_cols)
test_trt = encode_categoricals(test_trt, dummy_cols=["cp_time", "cp_dose"],
                               reference_columns=feature_cols)
X_test_raw = test_trt[feature_cols].values
print(f"Treated rows: {X_test_raw.shape[0]} | features: {X_test_raw.shape[1]}")
```

```
Treated rows: 3624 | features: 887
```

In [16]:
```
# Step 9.3 - apply the fold pipelines (transform only)
# Two feature variants, matching what each model was trained on (see Step 5):
# PCA features for the ResNet, raw RankGauss features for the FT-Transformer and CatBoost.
X_by_fold_pca = {fold: apply_fold_pipeline(X_test_raw, fold) for fold in range(N_FOLDS)}
X_by_fold_raw = {fold: apply_fold_pipeline_raw(X_test_raw, fold) for fold in range(N_FOLDS)}
```

In [17]:
```
# Step 9.4 - inference with the three models and weighted blend
time_start = time.time()
preds_test = predict_test_set_nn(X_by_fold_pca, X_by_fold_raw, num_targets=len(target_cols))
preds_test["catboost"] = predict_test_set_catboost(X_test_raw, num_targets=len(target_cols))
final_test_preds = sum(weights[name] * preds_test[name] for name in ALL_MODEL_NAMES)
print(f"[timing] Inference: {(time.time() - time_start) / 60:.1f} min")
```

```
[timing] Inference: 0.6 min
```

In [18]:
```
# Step 9.5 - build, check and export the submission
submission = build_submission(raw_test, sample_submission, target_cols,
                              final_test_preds, trt_mask_test)
sanity_check_submission(submission, sample_submission, raw_test, target_cols)

submission_path = f"{OUTPUT_DIR}/submission.csv"
submission.to_csv(submission_path, index=False)
print(f"{submission_path} written successfully ({submission.shape[0]} rows x {submission.shape[1]} columns).")
```

```
Min trt_cp probability: 1.44e-05 | Max: 9.99e-01 | Mean: 0.00371
Sanity check passed.
/kaggle/working/submission.csv written successfully (3982 rows x 207 columns).
```